DEVSECOPS LAB BOOK · PHASE 7

Chapter 27

Observability & Monitoring

Complete Logging & Metrics Stack

ELK Stack

Prometheus + Grafana

Jaeger Tracing

Wazuh SIEM

Time estimate: 5–6 hours

CHAPTER GOAL

Deploy a Complete Observability Stack

Centralized logging, metrics collection, alerting, and visualization across the entire platform.

ELK Stack deployed

Elasticsearch, Logstash, Kibana

Prometheus + Grafana

Metrics collection & dashboards

Distributed tracing

Jaeger across microservices

Centralized alerting

AlertManager notifications

SIEM integration

Wazuh security monitoring

Log aggregation

From all services

Real-time dashboards

Live visibility into systems

Alerts on anomalies

Before outages happen

Three Pillars of Observability

PILLAR 1: LOGGING

ELK Stack

- All app logs flow to Logstash
- Stored in Elasticsearch
- Searched in Kibana
- Debug issues by searching logs

PILLAR 2: METRICS

Prometheus + Grafana

- CPU, memory, disk from all nodes
- Application metrics (latency, errors)
- Scraped every 15 seconds
- Visualized in Grafana dashboards

PILLAR 3: TRACING

Jaeger

- Follow requests through microservices
- See where time is spent
- Find bottlenecks
- Performance optimization data

ALERTING (AlertManager)

Fires when metrics exceed thresholds · Notifies via Slack, email, PagerDuty · Automatic incident creation

27.2 ELK STACK

Centralized Logging Pipeline

Deployed on sec-obs-01 (10.0.2.30)

Elasticsearch

Single-node cluster storing indexed logs (logs-YYYY.MM.dd). Port 9200.

Logstash

Ingests via syslog (port 5000) & Kubernetes pods, parses JSON, ships to Elasticsearch.

Kibana

Web UI for searching & visualizing logs. Port 5601 — connects to Elasticsearch.

Retention flow (see 27.9)

Hot (0–7d)

Fast, fully indexed

Warm (7–30d)

Compressed, slower

Cold (30d+)

Archive, compliance

Metrics Collection & Visualization

Prometheus — scrapes every 15s

- Kubernetes nodes (role: node)
- Kubernetes pods (annotation-based)
- HAProxy — 10.0.2.25:8404
- Vault — 10.0.2.13:8200
- etcd — 3-node cluster (2379)

Rule files: alert_rules.yml evaluated every 15s and forwarded to AlertManager (9093).

Grafana — visualization layer

- Port 3000, default admin/admin (rotate immediately)
- Data source → Prometheus at http://prometheus:9090
- Persistent storage via grafana_storage volume
- Dashboards query recent + historical time-series data

Access

http://10.0.2.30:3000

System Metrics to Monitor

CPU Usage per Node

Alert	> 80% for 5 min
Alert	> 90% for 2 min
Critical	> 95%

Memory Usage per Node

Alert	> 85% for 5 min
Critical	> 95%

Disk Usage per Node

Alert	> 80% full
Alert	Unsustainable growth rate
Critical	> 95% full

Network I/O

Monitor	Bandwidth usage
Alert	Unexpected traffic spikes
Watch	DDoS patterns

Application & Cluster Health Signals

APPLICATION METRICS

Request Latency (p50/p95/p99)

Alert p95 > 1s (SLA) · Critical > 10s

Error Rate

Alert > 0.5% · Critical > 5%

Pod Restart Count

Frequent restarts indicate crashing containers

Service Availability

Alert on health-check / service down

KUBERNETES METRICS

Node Status

Alert NotReady · Critical: control-plane node down

Pod Status

Alert CrashLoopBackOff · Pending > 5 min

Scheduling

Critical: unable to schedule pods

Etcd Health

Alert member down · Critical: quorum lost

alert_rules.yml — Core Alerts

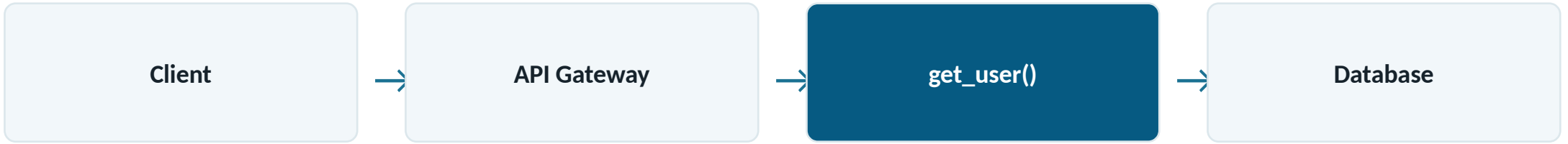
Evaluated by Prometheus, routed through AlertManager

Alert	Condition	For	Summary
NodeMemoryHigh	Available memory < 15%	5m	Node memory usage high
NodeDiskHigh	Available disk < 15%	5m	Node disk usage high
PodCrashLooping	Restarts > 5 in 30 min	—	Pod is crash looping
KubeletDown	up{job="kubelet"} == 0	5m	Kubelet on node is down
EtcdMemberDown	up{job="etcd"} == 0	5m	etcd member is down



Routing: AlertManager groups firing alerts and dispatches to Slack, email, or PagerDuty — with automatic incident creation for critical severities.

Jaeger — Follow a Request End to End



Each hop is a span carrying tags (e.g. user.id) — Jaeger stitches spans into one trace to reveal where time is spent.

Instrumentation snapshot (Flask + jaeger-client)

- Config: const sampler → sample every request; report to jaeger:6831
- tracer.start_active_span('get_user') wraps the request handler
- span.set_tag('user.id', user_id) attaches searchable context
- Access the Jaeger UI at http://10.0.2.30:16686

27.7 SIEM INTEGRATION

Wazuh — Security Event Monitoring

Agent deployed on every node

File Integrity Monitoring (FIM)

System configuration changes

Failed login attempts

Malware detection

Intrusion detection (HIDS)

Container activity

27.8 COMPLETE OBSERVABILITY DASHBOARD

Platform Health at a Glance

SYSTEM HEALTH

Nodes	**8 (all Ready)**
CPU / Mem	**42% / 58% avg**
Disk	**65% full**

APPLICATION HEALTH

Requests/sec	**5,000 (stable)**
Error rate	**0.02% (normal)**
Latency p99	**250ms (SLA OK)**

KUBERNETES CLUSTER

Control Plane	**3/3 healthy**
etcd	**Quorum OK**
Scheduled Pods	**120**

RECENT ALERTS

Critical	**0**
Warning	**1 (slow query)**
Firing > 1hr	**0**

SECURITY EVENTS (24H)

Failed logins	**3**
FIM changes	**0**
Malware alerts	**0**



ALL SYSTEMS NOMINAL

27.9 RETENTION POLICIES

How Long Data Sticks Around

LOG RETENTION (ELK)

Hot Tier

0–7 days

Searchable in Kibana, all logs indexed, fast queries.

Warm Tier

7–30 days

Searchable but slower, compressed storage, lower cost.

Cold Tier

30+ days

Archive storage, restore only if needed, compliance retention.

METRICS RETENTION

- Prometheus: 2-week default retention, fully configurable
- Storage footprint: ~15 GB per 1M metrics/day — plan capacity accordingly
- Grafana queries recent + historical time-series data in real time
- Adjust dashboard time range to reach further back into history

The Complete Toolchain

Tool	Purpose	Data Type
Elasticsearch	Log storage	Text logs
Logstash	Log processing	JSON / text
Kibana	Log visualization	Log searches
Prometheus	Metrics collection	Time-series data
Grafana	Metrics visualization	Dashboards
Jaeger	Distributed tracing	Request traces
AlertManager	Alerting	Metric thresholds
Wazuh	Security monitoring	Security events

KEY TAKEAWAY

See Everything

BEFORE OBSERVABILITY

- What's happening in the cluster?
- Why is the application slow?
- Is there a security issue?
- Data-driven decisions impossible

AFTER OBSERVABILITY

- Real-time visibility into all systems
- Logs, metrics, traces centralized
- Alerts fire before outages
- Security events detected
- Data-driven optimization